

Group 34 Report – CAB320

Assignment 1

Kes McLennan

Rory Blair

Riley Pederson

Section I – Important Features

In Artificial Intelligence, the state representation is crucial for defining the problem space of a system at any given moment. For Sokoban, this includes the positions of walls, boxes, the player, and target spots. The state can be static (immutable entities like walls and targets) or dynamic (mutable entities like the player and boxes). We represent static entities with tuples (due to their immutability) and dynamic entities with lists or arrays, allowing for modification during gameplay.

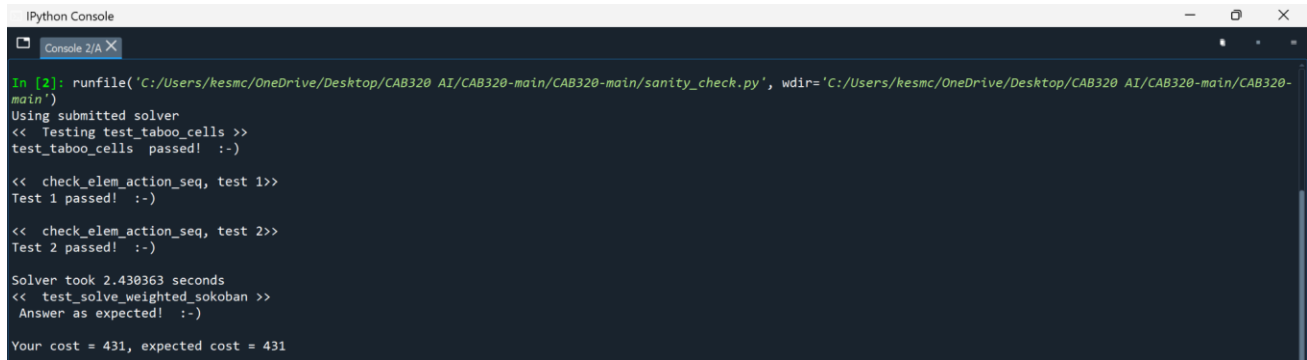
Static entities are in a state where the entity is immutable, meaning it cannot change its state. We have represented these states using tuples, which are immutable and ideal for representing fixed objects. Dynamic entities are in a state where the entity can interact and change with the environment. Dynamic entities are represented as lists or arrays as both allow for changes within and at certain points of the list/array. The dynamic entities used are the player and the boxes, as these entities constantly interact with the environment, and the two static entities are the walls and the target positions for the boxes, which are defined at the start of the game and will not change.

Our solver utilises the Manhattan Distance heuristic, which measures the minimum distance on a non-diagonal path between two points on a grid. Additionally, the solution is capable of detecting “deadlock” positions. Deadlock positions are considered a point where if a box was to end up, the puzzle would be deemed impossible to solve. These are specifically non-target corners and all points between two corners along a wall that aren’t targets are also considered deadlock points.

Another important feature is the search algorithms that we investigated, which are used in `solve_weighted_sokoban`. (search algorithms defined in `search.py`). The two different algorithms that were tested are the ‘`best_first_graph_search`’ and the ‘`astar_graph_search`’, which both require a problem and a heuristic. Both search algorithms are used to find an action sequence to the solution, and to calculate the total ‘cost’ of the solution. While A* search is optimal for finding the least-cost path due to its use of both path cost and heuristic, best-first search prioritises runtime, which may be at the expense of path optimality. Our chosen algorithm, `astar_graph_serach`, balances these considerations effectively, despite its higher computational demands and will consistently find the lowest cost to solve a puzzle if space allows.

Section II – Testing Methodology

How our code was first validated using the functions defined in `Sanity_Check`. `Taboo_cells`, `check_elem_action_seq`, and `solve_weighted_sokoban`, all passed their respective tests, as shown in Figure 1.



```

IPython Console
Console 2/A X
In [2]: runfile('C:/Users/kesmc/OneDrive/Desktop/CAB320 AI/CAB320-main/CAB320-main/sanity_check.py', wdir='C:/Users/kesmc/OneDrive/Desktop/CAB320 AI/CAB320-main/CAB320-main')
Using submitted solver
<< Testing test_taboo_cells >>
test_taboo_cells passed! :-)

<< check_elem_action_seq, test 1>>
Test 1 passed! :-)

<< check_elem_action_seq, test 2>>
Test 2 passed! :-)

Solver took 2.430363 seconds
<< test_solve_weighted_sokoban >>
Answer as expected! :-)

Your cost = 431, expected cost = 431

```

Figure 1: Results of preliminary testing using "Sanity_Check"

Following these preliminary tests, we expanded our testing to include a wide range of warehouse puzzles, noting significant variance in solve times, these ranges were from below 0.02 seconds to over two minutes.

Key observations from these tests revealed that puzzles with fewer boxes and more open space tend to solve faster. Conversely, puzzles with more boxes and constrained spaces (increased walls and taboo cells) took longer. For instance, warehouses like 8a, 47a, and 47b helped optimise weighted costs, while Warehouse 1 and 3 checked for valid moves and taboo cell avoidance. This structured approach allowed us to pinpoint inefficiencies and potential improvements in the solver. Warehouse 1 was also used to test `taboo_cells` to make sure the program was avoiding all taboo cells. (See Appendix 1, for all tested warehouse results) Below is a screenshot of a segment of the warehouse testing that was done. Times will vary depending on the specifications of the computer running the program.

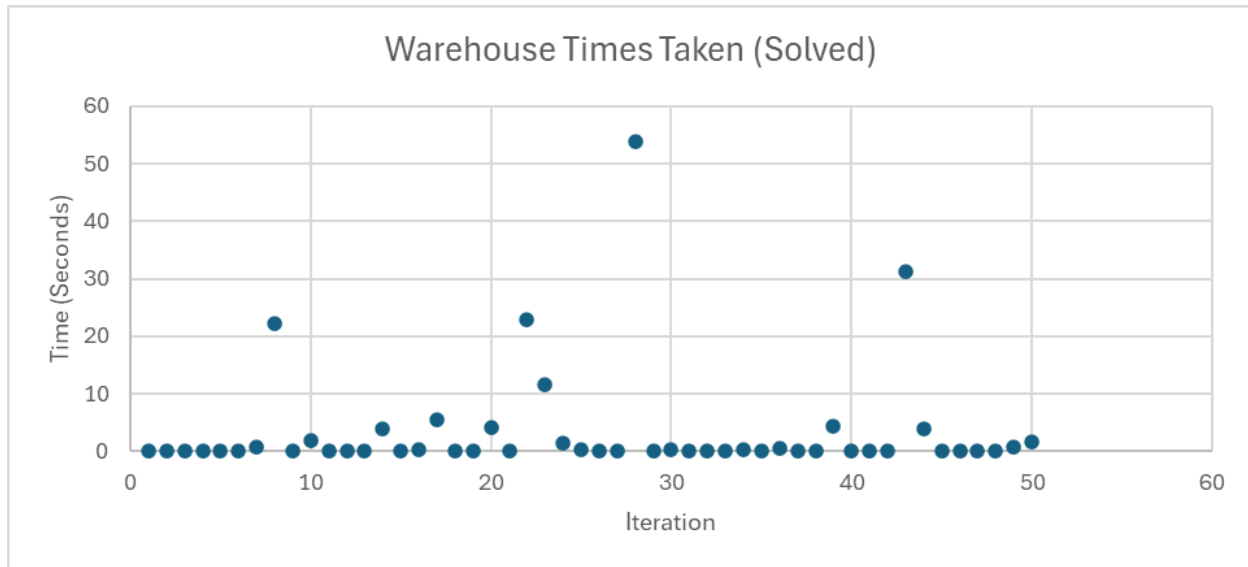


Figure 2: Solution Times through warehouses 1-99 (Only Solved)

Figure 2 shows a scatter plot representing the time taken to solve a set of warehouse puzzles (puzzles 1 - 99). The data illustrates that the vast majority of warehouses will be solved in predominately a matter of seconds, as shown by the large concentration of points near the 0 line. It should be noted that tests conducted (See appendix 1), were limited to a single minute of testing before being forced to timeout. The outliers, such as the points corresponding to the times above 30 seconds, indicate warehouses where the solver's performance deviated from the norm due to increased complexity or less optimal paths taken. This does suggest that although for less complex puzzles, the algorithm is exceptionally efficient. The second puzzles increase in complexity, it dictates how important it would be in future to introduce some changes and refine the heuristic or use a more advanced a* search algorithm.

Warehouse File	Solver Time (s)	Result	Cost
warehouse_01.txt	0.015008	Solved	33
warehouse_01_a.txt	0.013012	Solved	73

Table 1: Weighted warehouse vs Unweighted results

It should also be noted that the heuristic is the same for both weighted and non-weighted puzzles. Table 1 shows a test run between the same puzzle with the only parameter changing being the weight of the boxes. In the weighted scenario, the solve time is marginally lower at 0.013012 seconds compared to 0.015008 seconds for the unweighted puzzle. Although there is a slight decrease in time which could suggest the algorithm is more suited for weighted puzzles, and may suggest that it can optimise for certain constraints or rules (such as those applied with weighting), however, this would be unlikely and, it's more likely that the puzzle was finished just as easily due to the lack of complexity in the space and more testing should be introduced surrounding weighted puzzles.

Section III – Performance and Limitations

Our tests covered a broad range of warehouse configurations with varied complexities, and the performance of our Sokoban solver demonstrated both its capabilities and limitations. Most warehouses were solved in less than 0.5 seconds, and many, such as Warehouse 9, were resolved in just 0.02 seconds. However, more complex scenarios presented challenges, with Warehouse 13 taking 36.8 seconds and Warehouse 33 exceeding two minutes to solve. These results highlight the need for further optimisations to enhance efficiency and manage space-time trade-offs more effectively.

The A* graph search algorithm, while robust, tends to use space exponentially, which becomes problematic in denser puzzle configurations. Implementing an iterative-deepening A* search could address this by balancing the space cost against computational time, particularly beneficial in puzzles requiring deep solution searches. Additionally, it may be possible to use GPU-acceleration or multi-threading and leverage these parallel computing techniques to give the appearance of a more efficient algorithm, while in reality, using more computational power to solve the puzzles more efficiently.

Moreover, the Manhattan distance heuristic, occasionally will compromise solution accuracy for computational speed. This heuristic might be too basic for these scenarios, especially in tightly packed puzzles where a more precise estimation of costs could expedite the solving process. A refined approach might involve detailed calculations of all possible box-to-target permutations to provide a closer approximation to the actual solution cost. This method, however, scales poorly as the number of boxes increases, necessitating a careful consideration of heuristic precision against the exponential rise in computational demands. Such strategic enhancements are crucial for advancing the solver's performance, ensuring it remains both fast and reliable even as puzzle complexity increases. Alternate heuristics could also be considered such as Euclidean distance or the Chebyshev distance algorithm or developing more advanced heuristics with better capabilities of solving advanced scenarios or the iterative-deepening A* search, reducing the space complexity that the current heuristic cannot deal with appropriately.

This potential improvement could potentially construct a Sokoban Solver capable of solving significantly more puzzles as only 32/56 puzzles tested (Appendix 1) would be capable of solving before the one-minute timeout.

Appendix

Appendix 1 – Results of warehouses 1-99 (Timeout of 60 seconds)

Warehouse File	Solver Time (s)	Result	Cost
warehouse_01.txt	0.015008	Solved	33
warehouse_01_a.txt	0.013012	Solved	73
warehouse_03.txt	0.101016	Solved	41
warehouse_03_impossible.txt	0.022658	Impossible	None
warehouse_07.txt	Timeout	Skipped	-
warehouse_09.txt	0.009082	Solved	396
warehouse_11.txt	0.793762	Solved	78
warehouse_13.txt	22.206843	Solved	52
warehouse_15.txt	0.033522	Solved	37
warehouse_17.txt	1.915856	Solved	26
warehouse_19.txt	0.054113	Solved	41
warehouse_21.txt	0.015015	Solved	17
warehouse_23.txt	0.025541	Solved	56
warehouse_25.txt	3.993393	Solved	29
warehouse_27.txt	0.045536	Solved	50
warehouse_29.txt	0.38471	Solved	104
warehouse_31.txt	5.558099	Solved	17
warehouse_33.txt	Timeout	Skipped	-
warehouse_35.txt	Timeout	Skipped	-
warehouse_37.txt	4.147301	Solved	71
warehouse_39.txt	0.184664	Solved	85
warehouse_41.txt	22.980897	Solved	50
warehouse_43.txt	11.52474	Solved	61
warehouse_45.txt	1.395167	Solved	45
warehouse_47.txt	0.357633	Solved	179
warehouse_49.txt	Timeout	Skipped	-
warehouse_51.txt	0.092017	Solved	34
warehouse_53.txt	53.833719	Solved	37
warehouse_55.txt	0.118148	Solved	64
warehouse_57.txt	0.240245	Solved	60
warehouse_59.txt	Timeout	Skipped	-
warehouse_5n.txt	Timeout	Skipped	-

Group 34

warehouse_61.txt	Timeout	Skipped	-
warehouse_63.txt	0.386442	Solved	101
warehouse_65.txt	Timeout	Skipped	-
warehouse_67.txt	0.584791	Solved	37
warehouse_69.txt	Timeout	Skipped	-
warehouse_6n.txt	Timeout	Skipped	-
warehouse_71.txt	4.32842	Solved	120
warehouse_73.txt	Timeout	Skipped	-
warehouse_75.txt	Timeout	Skipped	-
warehouse_77.txt	Timeout	Skipped	-
warehouse_79.txt	31.289994	Solved	48
warehouse_81.txt	4.032006	Solved	376
warehouse_83.txt	Timeout	Skipped	-
warehouse_85.txt	Timeout	Skipped	-
warehouse_87.txt	Timeout	Skipped	-
warehouse_89.txt	Timeout	Skipped	-
warehouse_8a.txt	0.676112	Solved	431
warehouse_8b.txt	1.647727	Solved	27
warehouse_91.txt	Timeout	Skipped	-
warehouse_93.txt	Timeout	Skipped	-
warehouse_95.txt	Timeout	Skipped	-
warehouse_97.txt	Timeout	Skipped	-
warehouse_99.txt	Timeout	Skipped	-